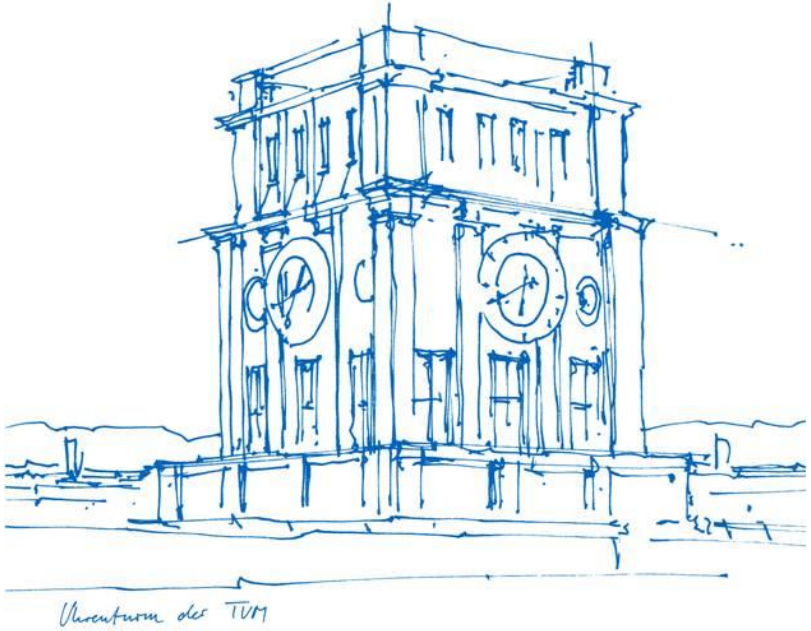
Lehrstuhl für Rechnerarchitektur & Parallele Systeme

Grundlagenpraktikum: Rechnerarchitektur

Sobel-Filter (A201)

Salim Daoud - Tobias Langer - Luca Tänzler



Technische Universität München
TUM School of Computation, Information and Technology
Wintersemester 24/25

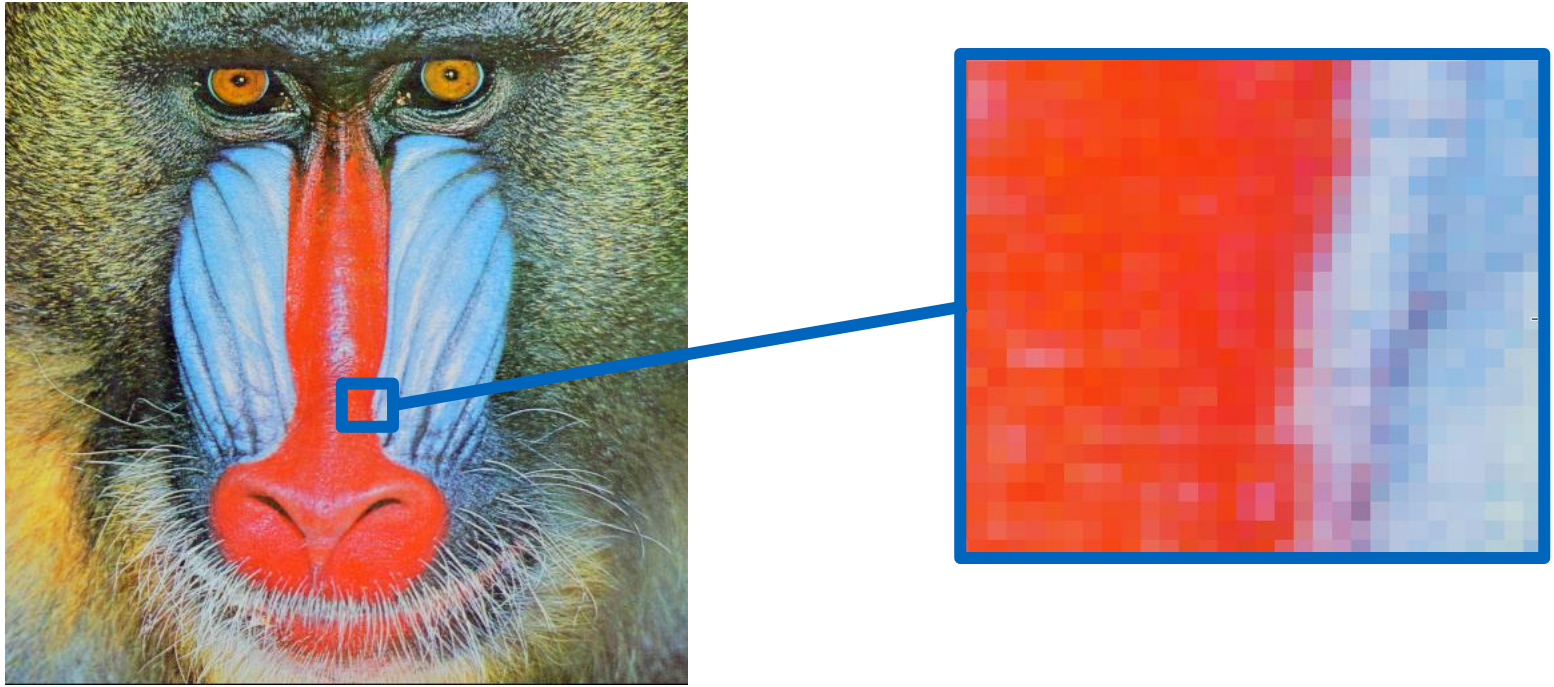
Content

1. Problem Statement
2. Solutions and Optimizations
3. Correctness
4. Performance Analysis
5. Summary and Outlook

Implement edge detection using Sobel-Filter in C

- Grayscale Conversion
- Edge Detection with Sobel-Filter
- Optimize Computation
- Performance Evaluation

Detecting edges



Detecting edges

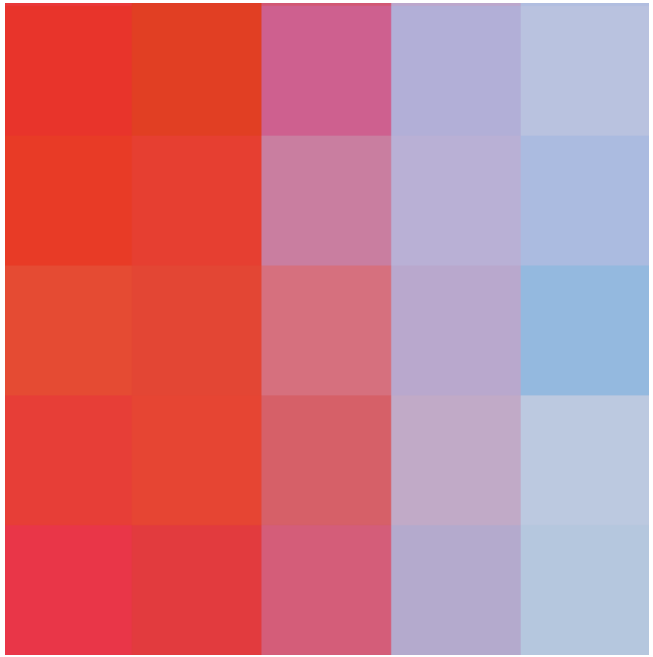
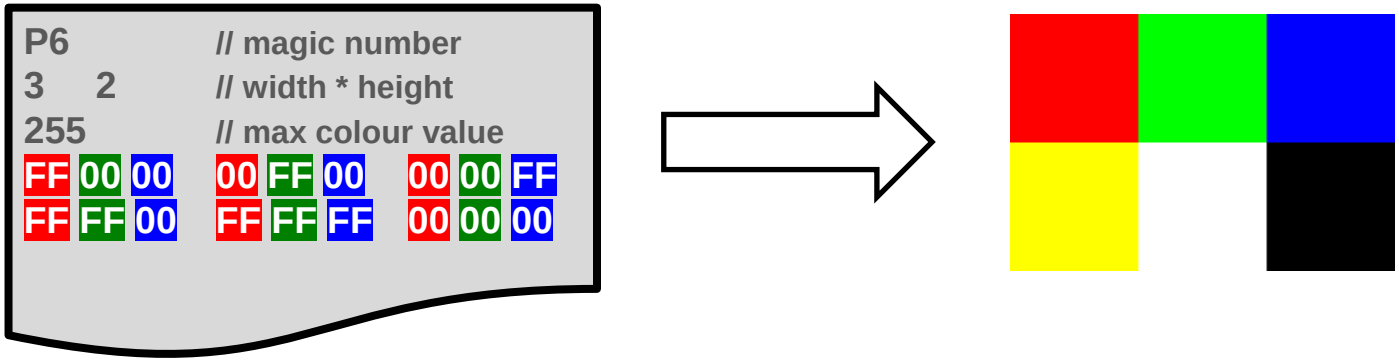


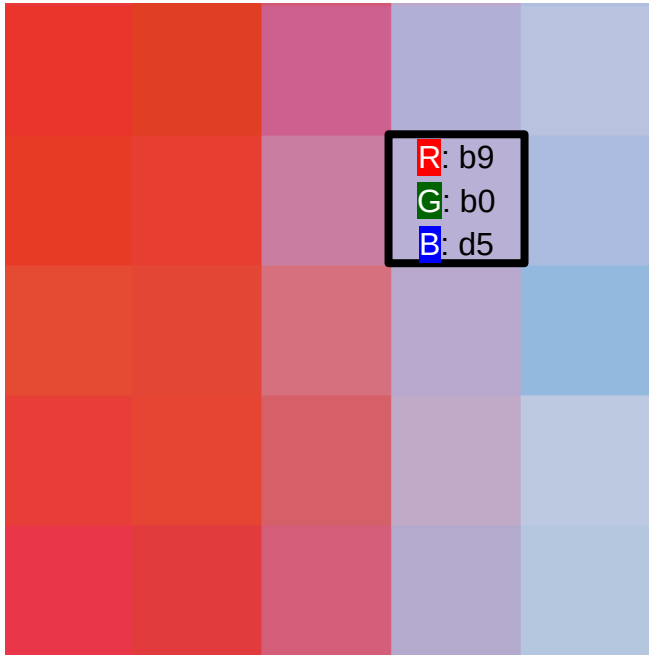
Image Format: PPM & PGM

PPM is used for storing color images in simple, binary format:



PGM is used for grayscale images, storing only one intensity value per pixel instead of three.

Grayscaling



Basic: mean value

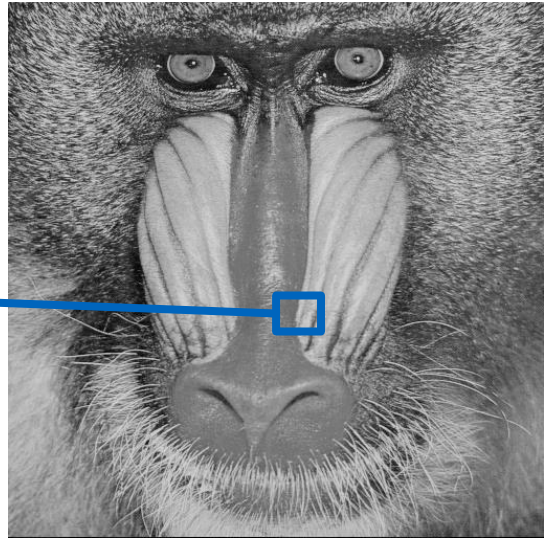
$$\boxed{G = \frac{R+G+B}{3}}$$

Adapted to the human eye: weighted values:

$$\boxed{G = R * 0.299 + G * 0.587 + B * 0.114}$$

Grayscaling

104	108	134	180	194
108	111	152	182	186
118	114	144	177	178
111	115	132	180	199
111	109	131	176	196



Sobel Filter - Formal Description

104	108	134	180	194
108	111	152	182	186
118	114	144	177	178
111	115	132	180	199
111	109	131	176	196

2 Sobel Kernels:

$$M^v = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{pmatrix} \qquad M^h = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

Convolution:

$$Q^d_{(x,y)} = \sum_{i=-1}^{1} \sum_{j=-1}^{1} M^d_{(1+i,1+j)} \cdot Q_{(x+i,y+j)} \qquad d \in \{v, h\}$$

Combining Q^v and Q^h

$$Q'_{(x,y)} = clamp_0^{255}\left(\sqrt{\left(Q^v_{(x,y)}\right)^2 + \left(Q^h_{(x,y)}\right)^2}\right)$$

Naive Implementation

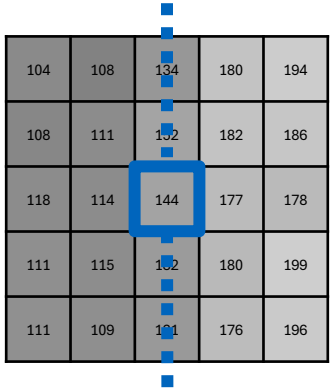
104	108	134	180	194
108	111	132	182	186
118	114	144	177	178
111	115	132	180	199
111	109	131	176	196

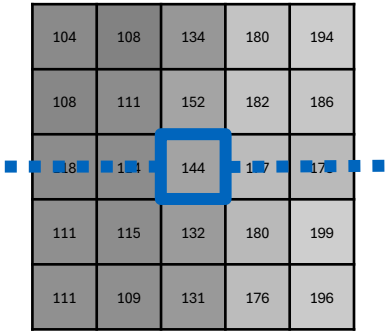
Applying Vertical Kernel:

```
int kern_vertical[3][3] = {    {1, 0, -1},
                               {2, 0, -2},
                               {1, 0, -1} };
```

111 + 2 * 114 + 115
+ 0 * 152 + 0* 144 + 0* 132 = - 262
- (182 + 2 * 177 + 180)

Naive Implementation



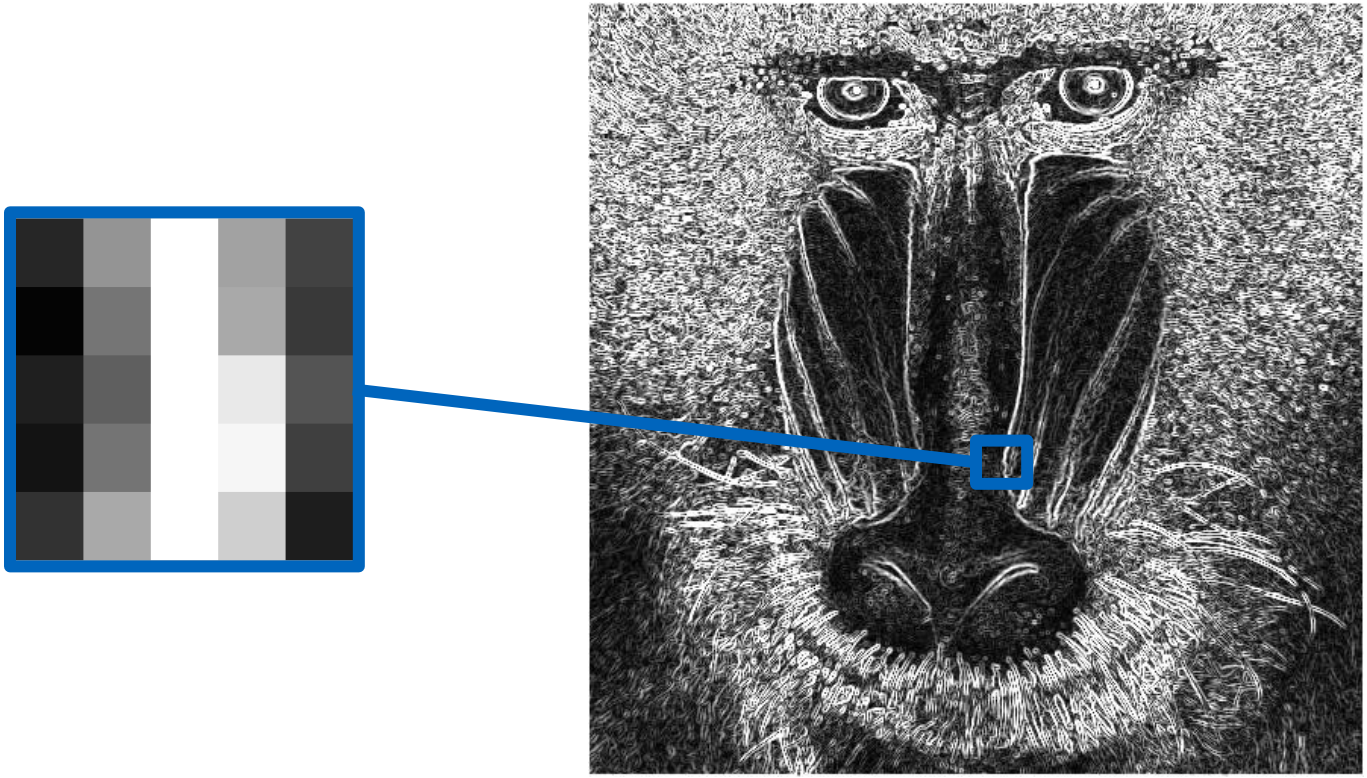



```
int magnitude = (int) sqrt(sum_vertical * sum_vertical + sum_horizontal * sum_horizontal);
```

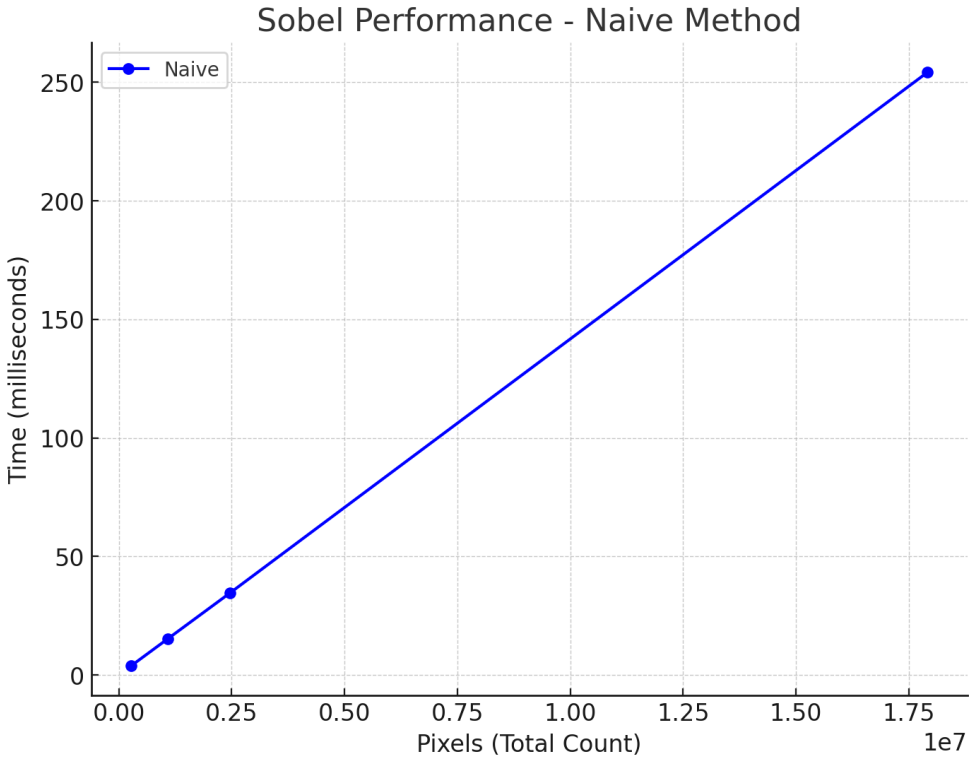
Result

38	148	255	162	66
4	117	255	169	57
31	95	255	233	84
19	116	255	246	63
50	169	255	207	29

Result



Sobel naive performance



AMD Ryzen 5 4680U, 2.20 GHz, 8 GB RAM, compiled with -O2

Sobel Kernel Unroll

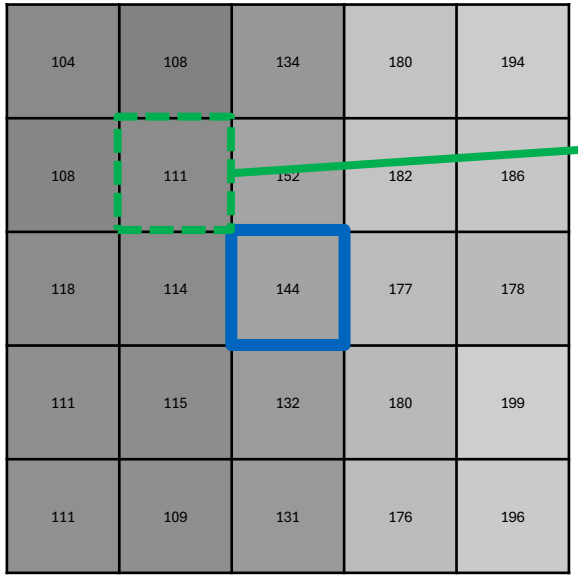
$$M^v = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{pmatrix}$$

$$M^h = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

Iteration over 2 Kernels to fetch scale factors is inefficient, but:

- fixed, small size
- both Kernels address same same pixels
- multiplication with 0

Sobel Kernel Unroll



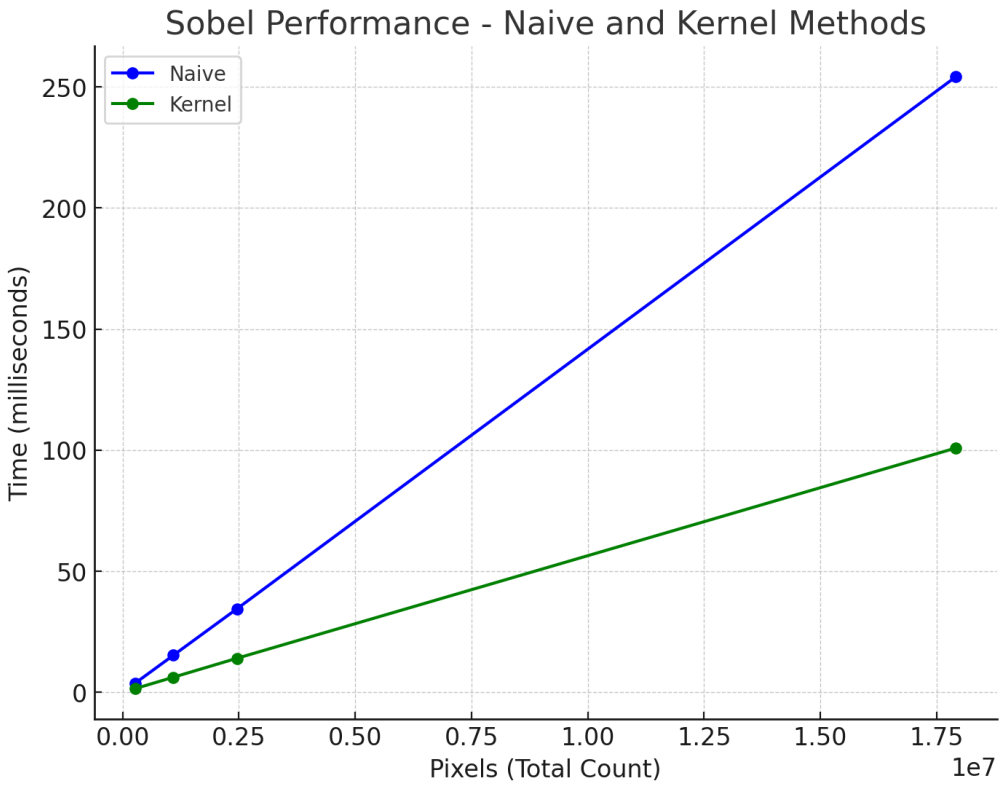
buffer and reuse neighbouring pixels

```
uint8_t top_left =  image_row_prev[gray_x - 1];
```

hardcode formula per Kernel

```
gradient_vert = top_left + left + left + bot_left
                - top_right - right - right - bot_right;
```

Sobel Kernel Unroll performance



60% faster than Naive

AMD Ryzen 5 4680U, 2.20 GHz, 8 GB RAM, compiled with -O2

Sobel Separate Convolution

right neigbour:

$$[\ldots] - \begin{bmatrix} 1 * 152 \\ 2 * 144 \\ 1 * 132 \end{bmatrix}$$

104	108	134	180	194
108	111	152	182	186
113	114	144	177	178
111	115	132	180	199
111	109	131	176	196

left neighbour:

$$[\ldots] + \begin{bmatrix} 1 * 152 \\ 2 * 144 \\ 1 * 132 \end{bmatrix}$$

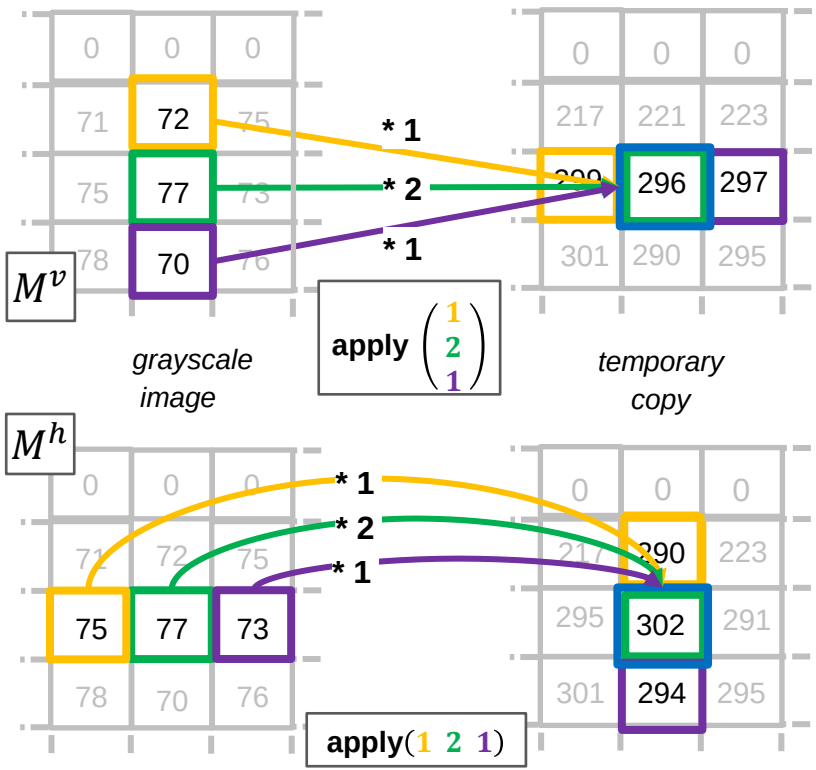
Sobel Separated Convolution

- The Sobel filter matrix is separable.
- Horizontal and vertical kernel can be displayed as the outer product of two 1-dimensional vectors:

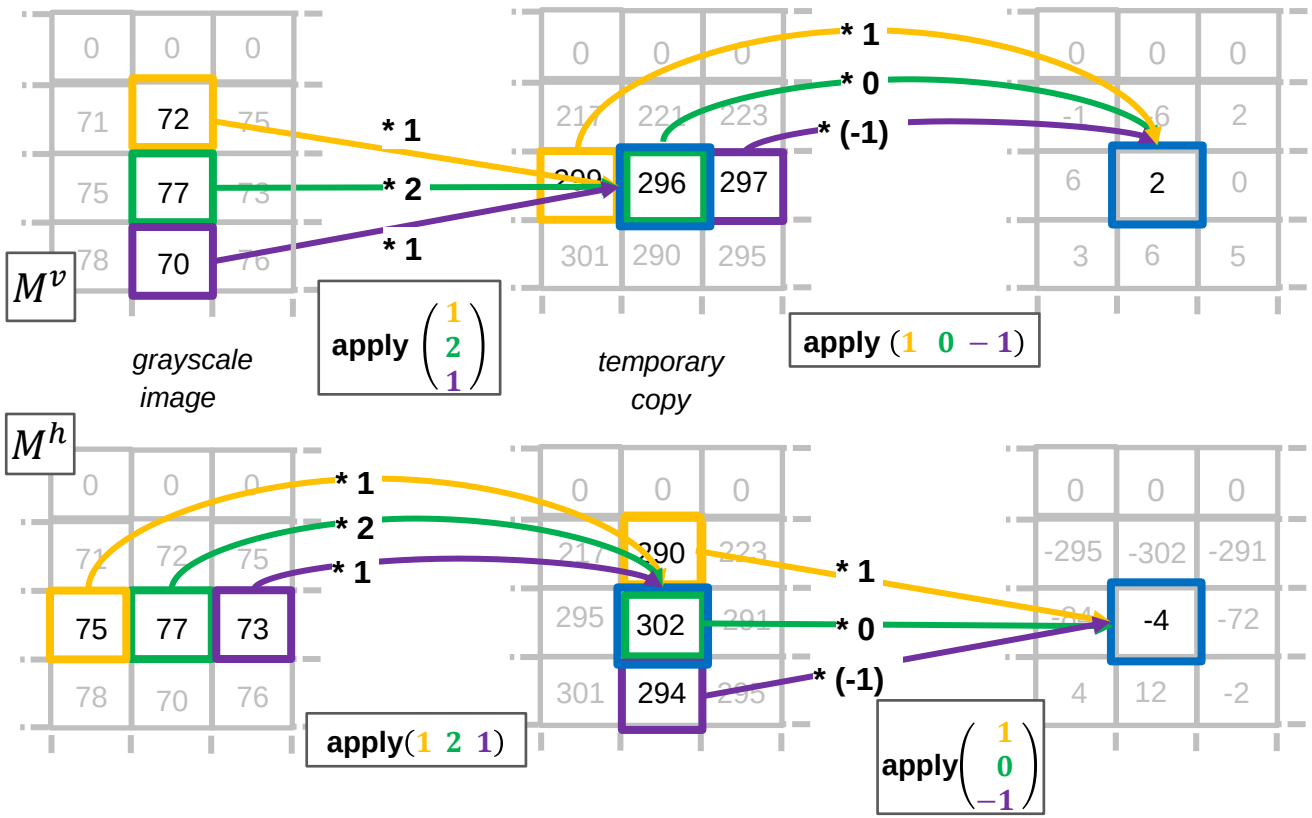
$$M^v = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} * \begin{pmatrix} 1 & 0 & -1 \end{pmatrix}$$

$$M^h = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \end{pmatrix} * \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}$$

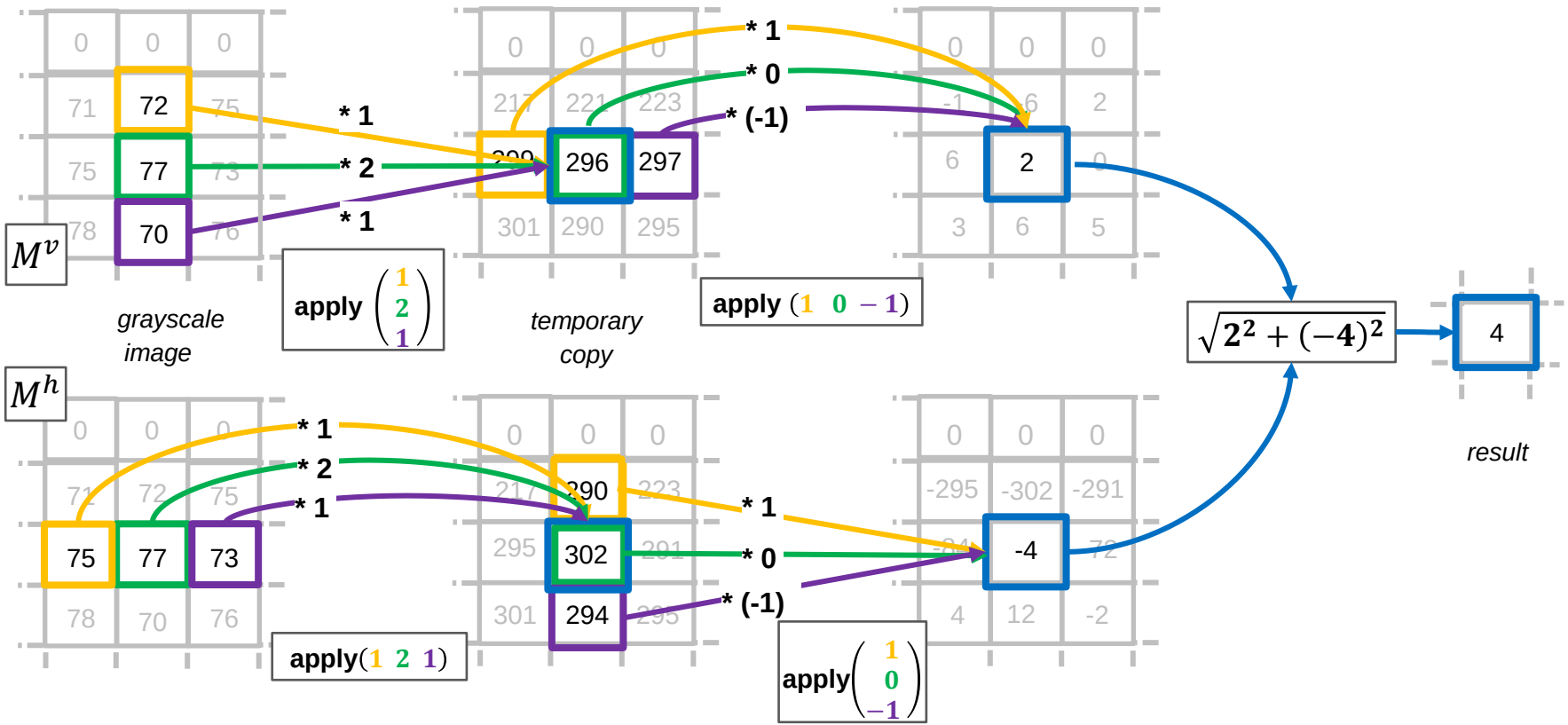
Sobel Separate Convolution



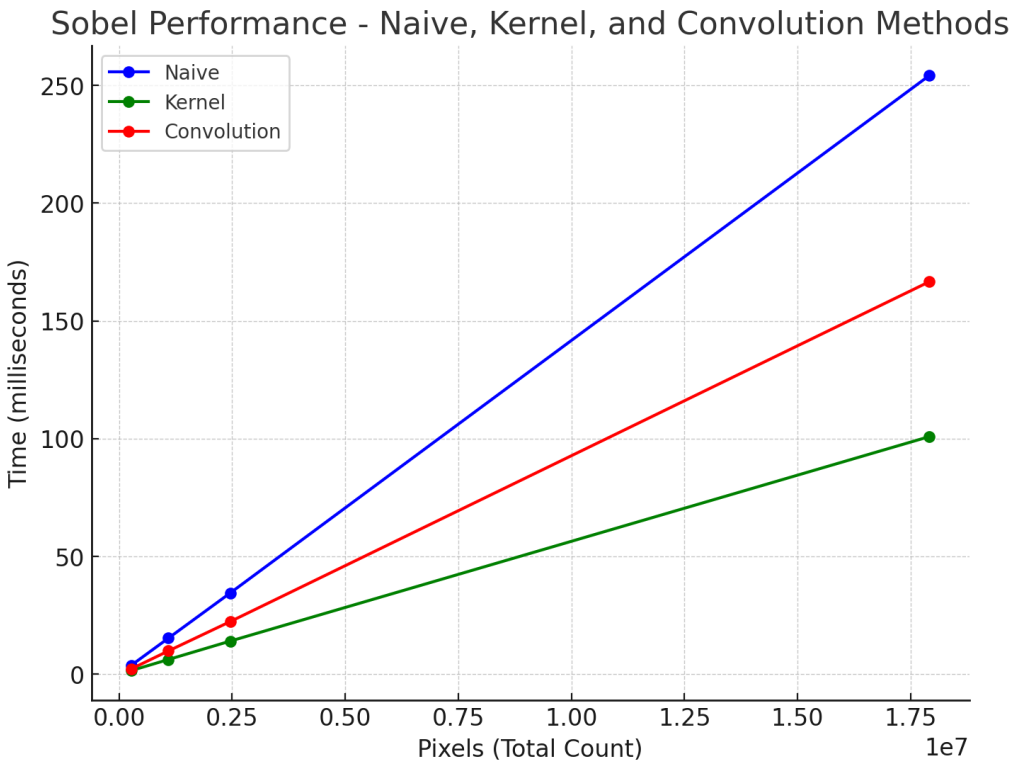
Sobel Separate Convolution



Sobel Separate Convolution



Sobel Seperated Convolution performance

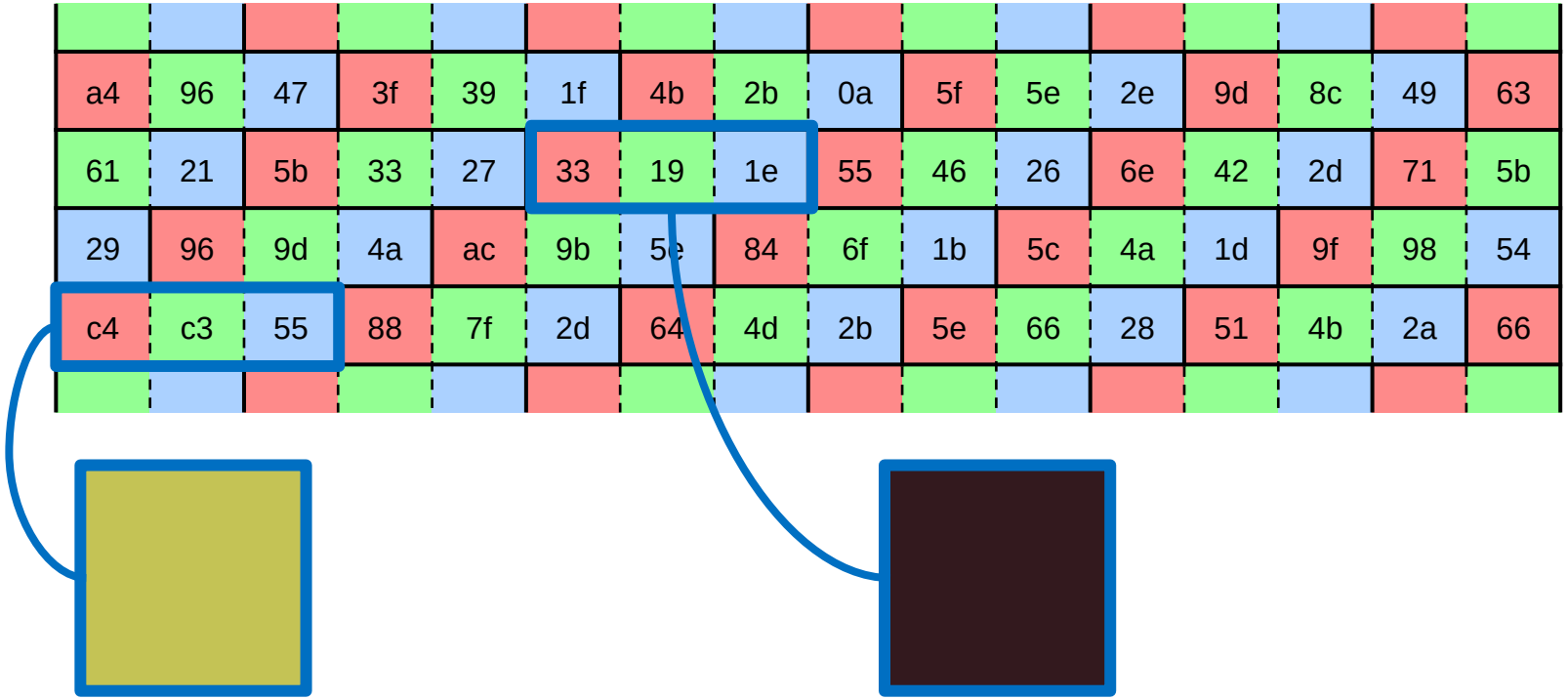


34% faster than Naive

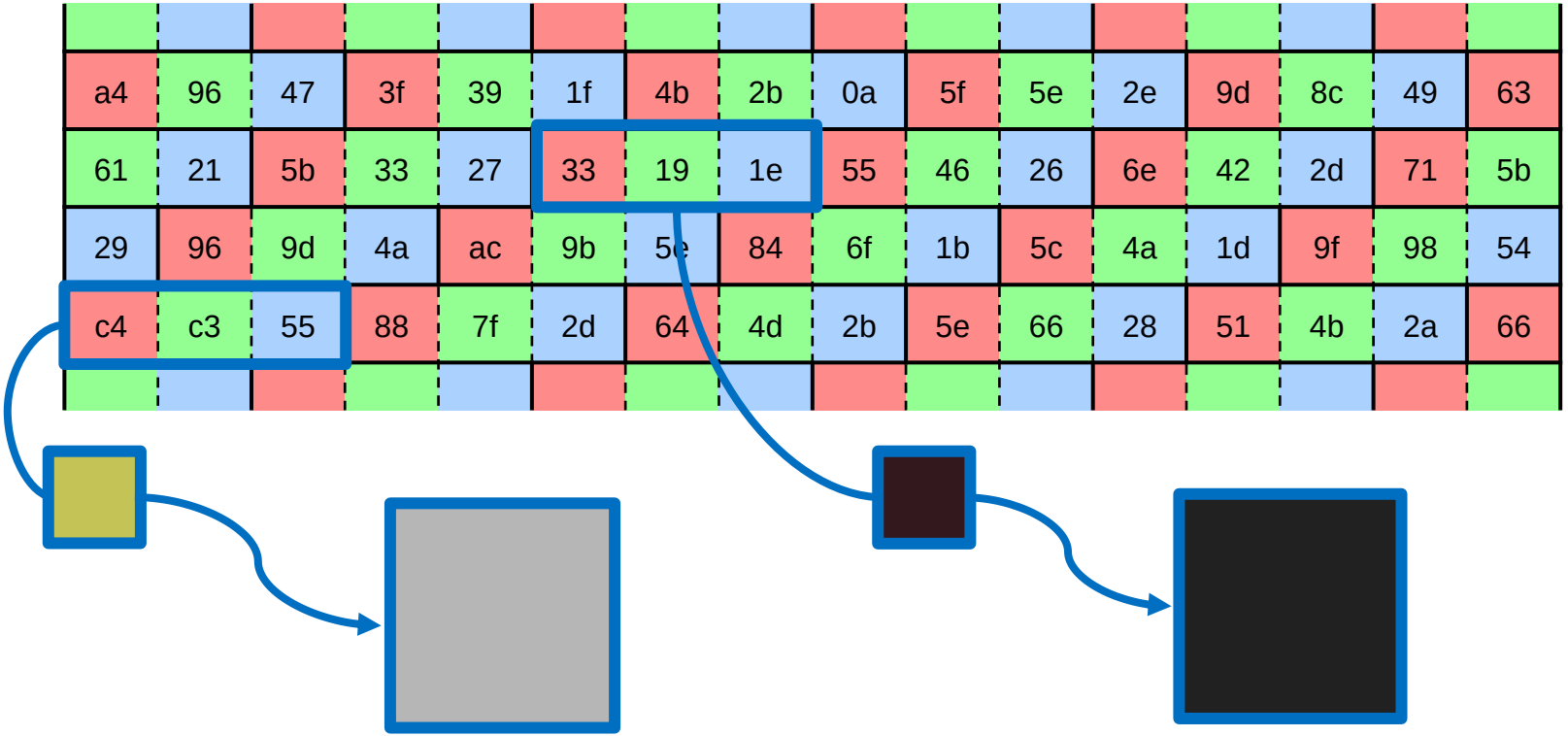
But 39% slower than Kernel unroll

AMD Ryzen 5 4680U, 2.20 GHz, 8 GB RAM, compiled with -O2

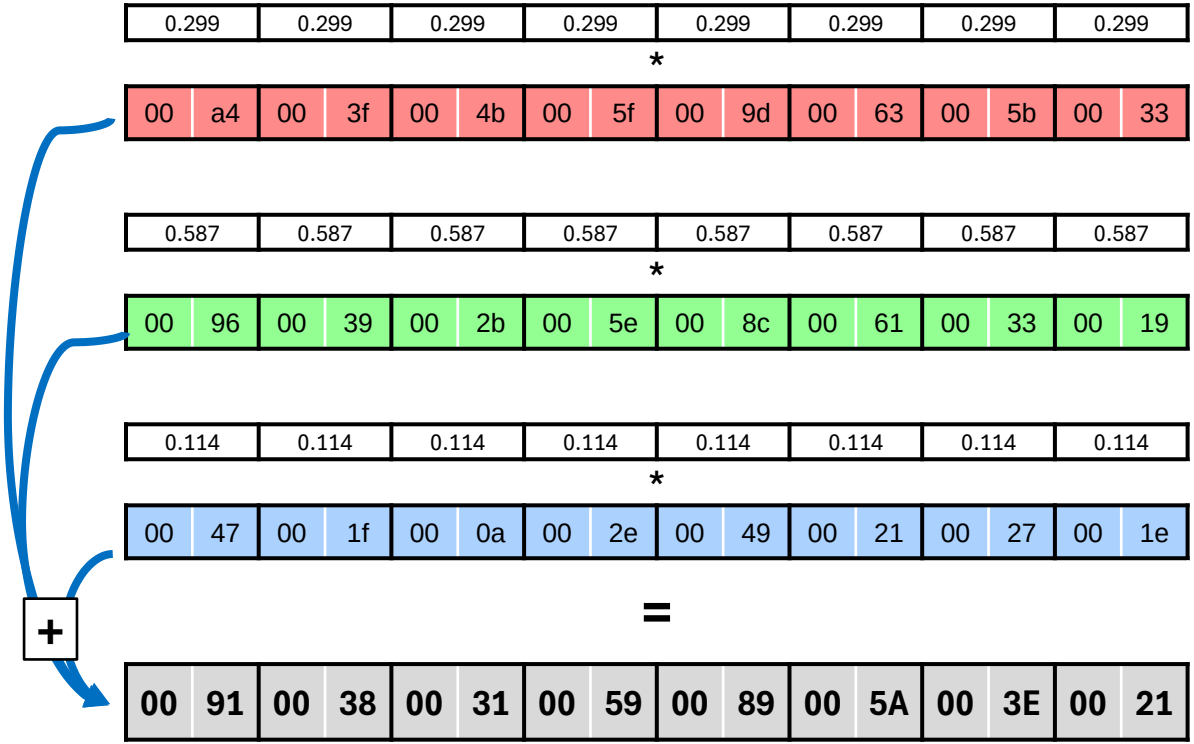
RGB to Grayscale



RGB to Grayscale



RGB to Grayscale SIMD – basic idea



Obstacles:

- Floating point multiplication needs 32 Bit
- RGB values not stored as a sequence

Grayscaling 8 pixels with SIMD

a4	96	47	3f	39	1f	4b	2b	0a	5f	5e	2e	9d	8c	49	63
61	21	5b	33	27	33	19	1e	55	46	26	6e	42	2d	71	5b
29	96	9d	4a	ac	9b	5e	84	6f	1b	5c	4a	1d	9f	98	54
c4	c3	55	88	7f	2d	64	4d	2b	5e	66	28	51	4b	2a	66

_mm_loadu_si128((m128i*)(image_pointer))

reg1:

5f	5e	2e	9d	8c	49	63	61	21	5b	33	27	33	19	1e	55
0			1			2			3			4		

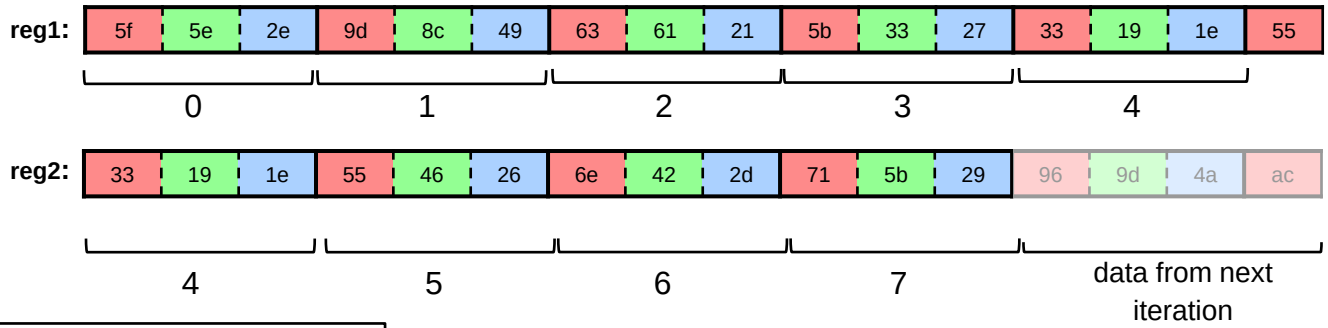
_mm_loadu_si128((m128i*)(image_pointer + 12))

reg2:

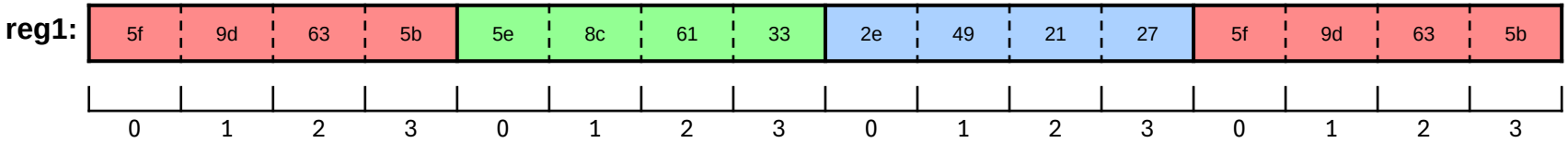
33	19	1e	55	46	26	6e	42	2d	71	5b	29	96	9d	4a	ac
4			5			6			7			data from next iteration			

8 values can be computed using 1 *__m128i* register and fixed point arithmetic.

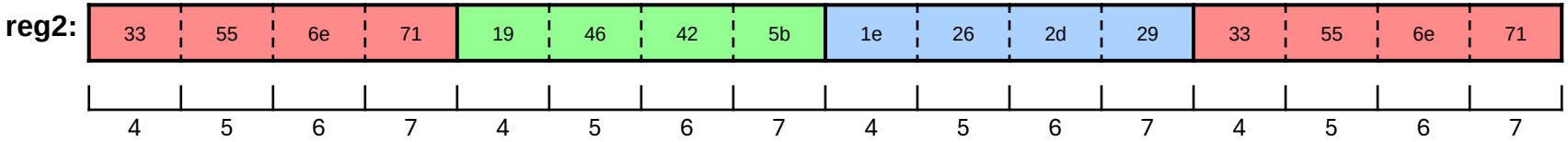
Grouping the RGB data



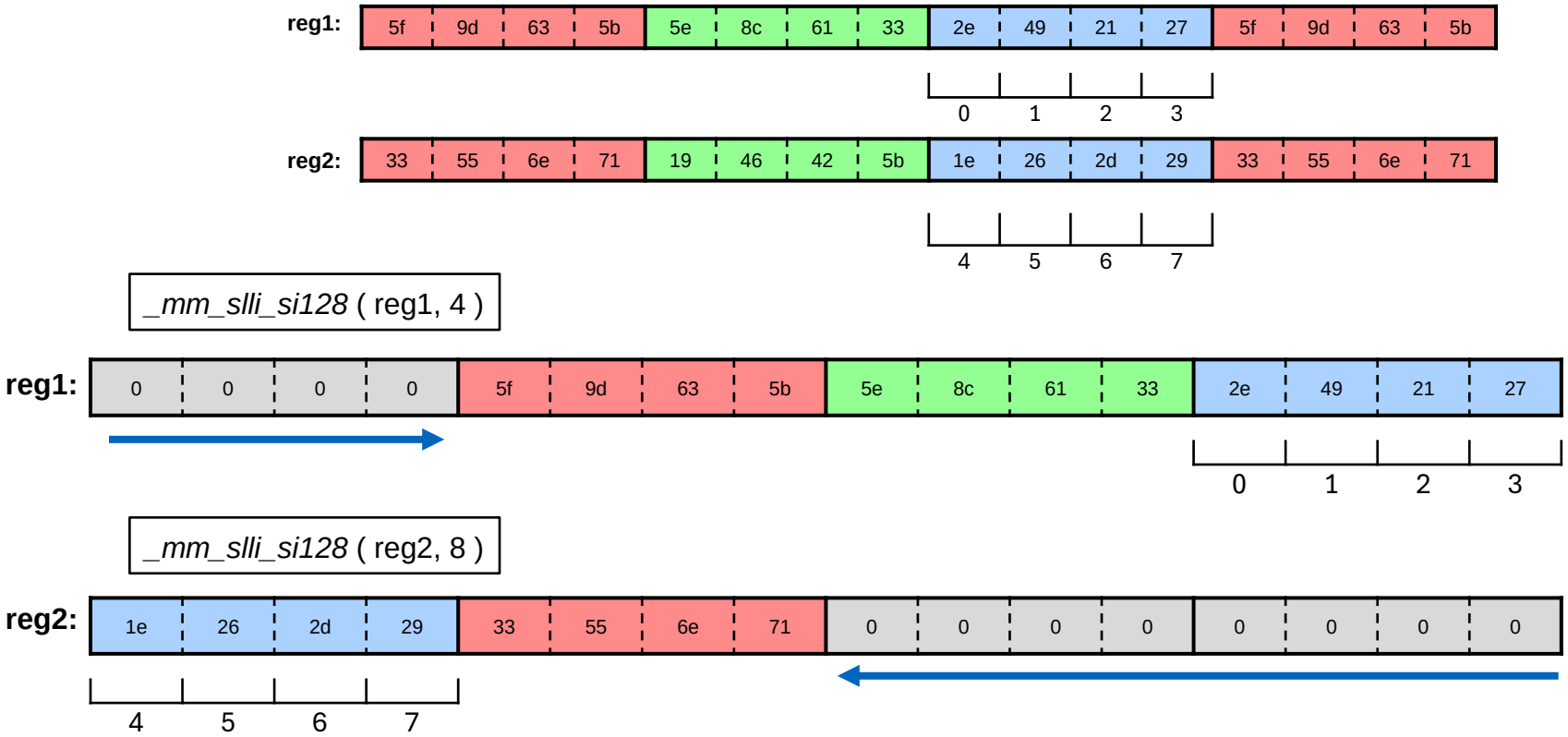
_mm_shuffle_epi8(reg1, mask_0123)



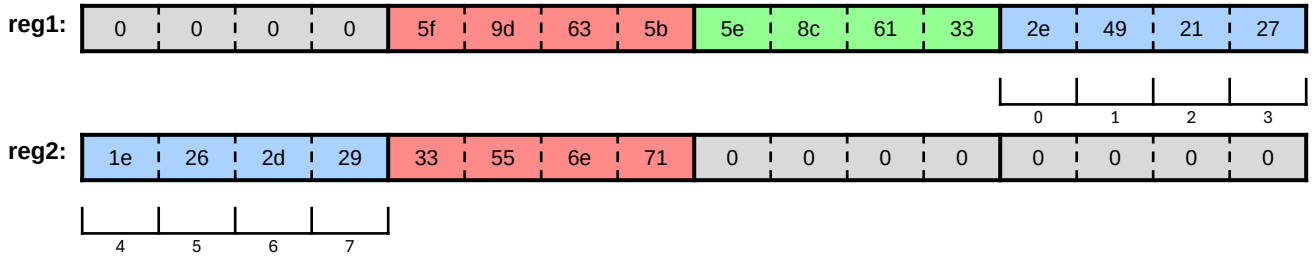
_mm_shuffle_epi8(reg2, mask_4567)



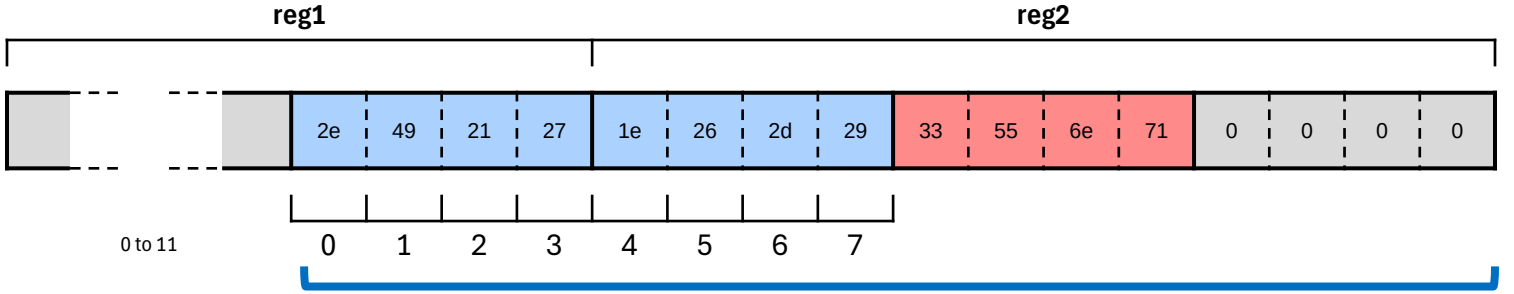
Isolating single colours



Extracting single colours

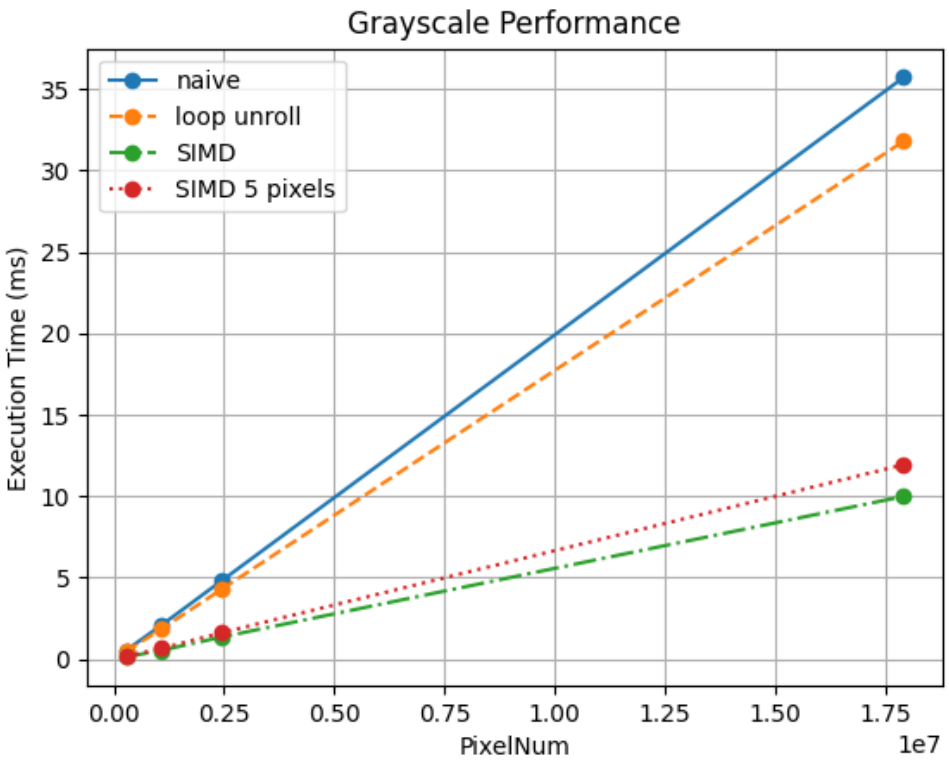


_mm_alignr_epi8(reg1, reg2, 12)



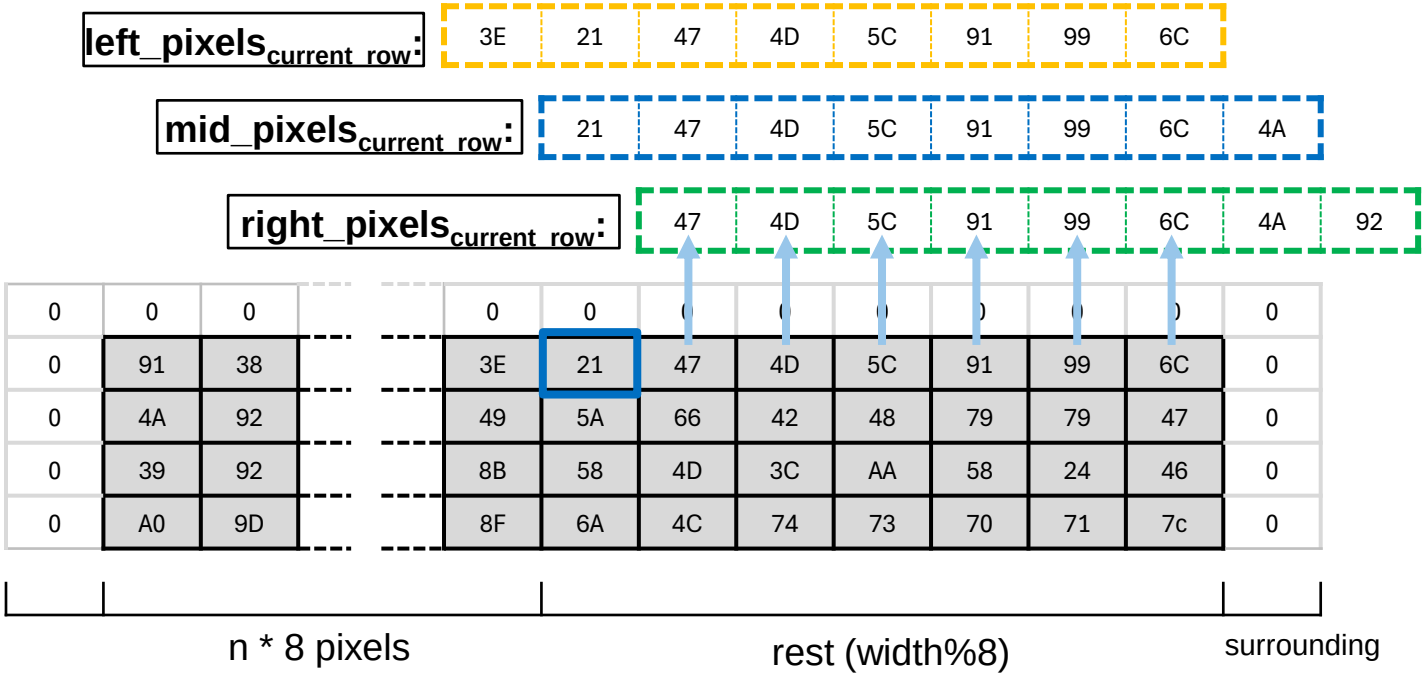
extend to 16 Bit values and apply respective coefficient

Grayscaling performance overview

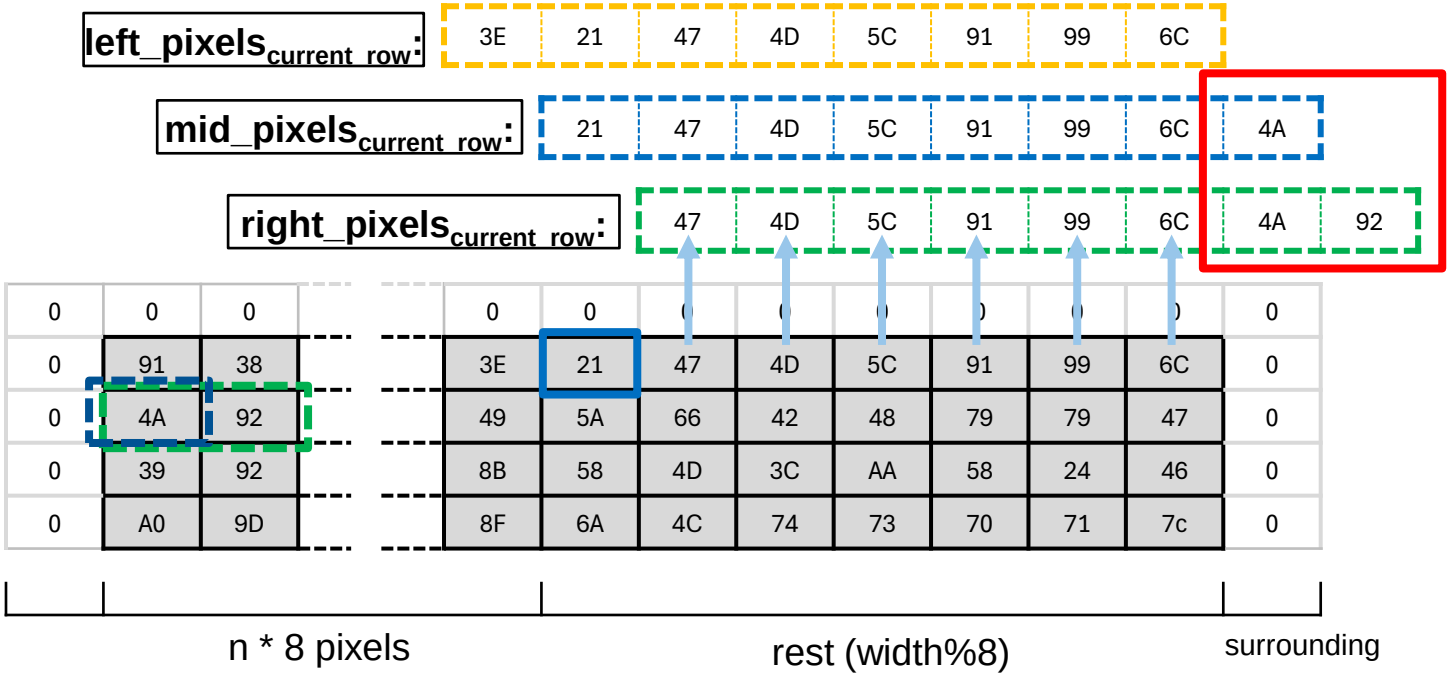


compiled with -O2

Sobel SIMD


left_pixels current_row:
3E 21 47 4D 5C 91 99 6C
mid_pixels current_row:
21 47 4D 5C 91 99 6C 4A
right_pixels current_row:
47 4D 5C 91 99 6C 4A 92
0 0 0 0 0 0 0 0 0 0 0 0 0
0 91 38 3E 21 47 4D 5C 91 99 6C 0
0 4A 92 49 5A 66 42 48 79 79 47 0
0 39 92 8B 58 4D 3C AA 58 24 46 0
0 A0 9D 8F 6A 4C 74 73 70 71 7c 0
n * 8 pixels
rest (width%8)
surrounding

Sobel SIMD



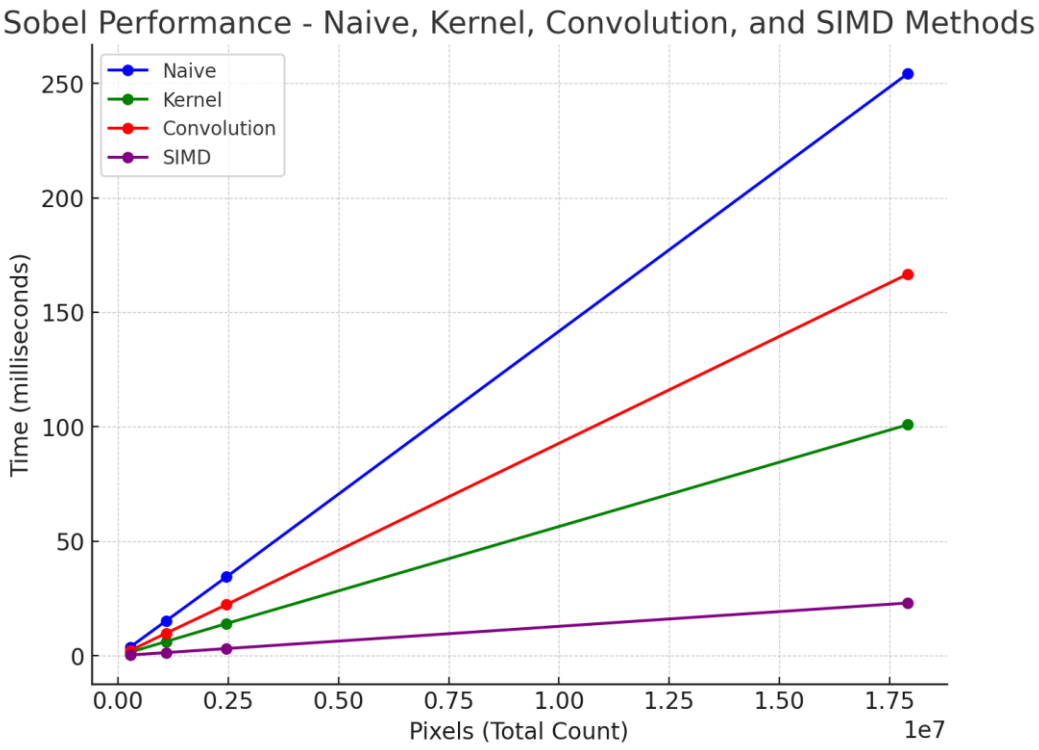
Problem:

loading 8 values at once leads to overlapping into the next rows

Solution:

Masking the loaded values depending on the number of remaining pixels per row.

Sobel SIMD performance



90% faster than Naive

77% faster than Kernel unroll

86% faster than Convolution

AMD Ryzen 5 4680U, 2.20 GHz, 8 GB RAM, compiled with -O2

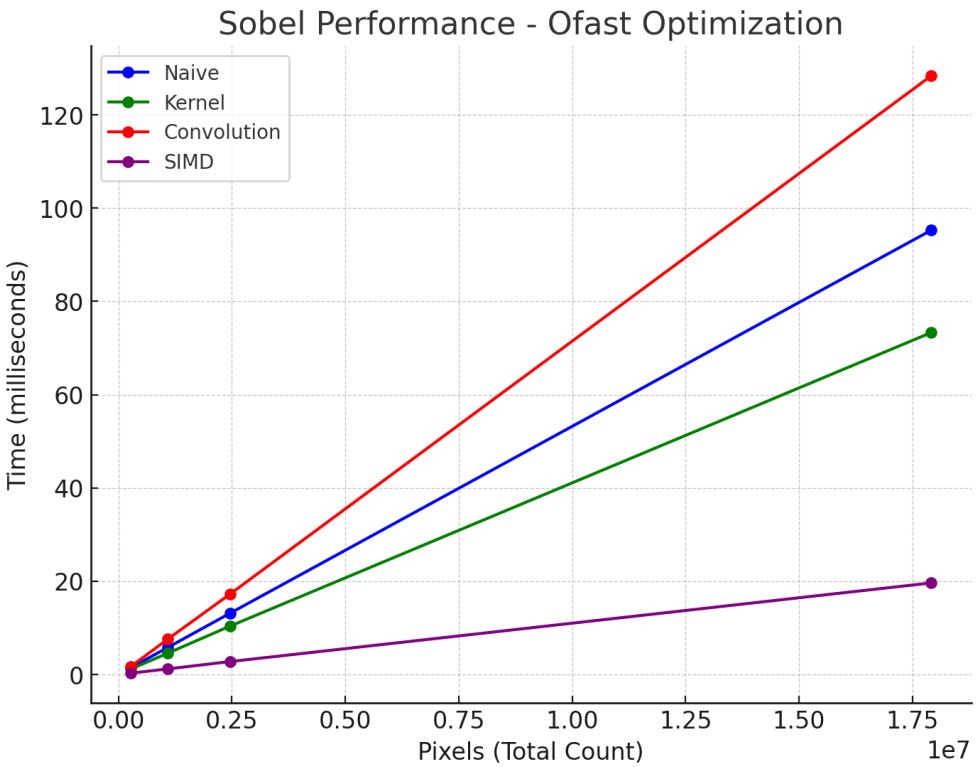
Unit tests

```
==========================================================================================
SUCCESSFUL:  Grayscale SIMD Basic Test
==========================================================================================
SUCCESSFUL:  Grayscale Basic Test
==========================================================================================
WARNING: The grayscale coefficients are only roughly approximated: Red (0.25), Green (0.5) Blue (0.25).
Given coefficients for Red (0.299), Green (0.587) and Blue (0.114) will not be used.
FAILED:      Grayscale Bitshift Basic Test
             Result does not match expectations, see test_img_to_grayscale_bitshift() in test/grayscale_test.c
             Details:
                          Expected:    Got:
                     [ 0]       203     189
                     [ 1]       165     154
                     [ 2]       117     120
                     [ 3]       134     151
                     [ 4]        97     113
                     [ 5]       164     150
                     [ 6]       128     119
                     [ 7]       167     145
                     [ 8]       211     215
                     [ 9]        97     114
```

Frameworkless unit tests for:

- ReadWrite
- Grayscaling
- Sobel

Sobel performance with Ofast



62% faster for Naive

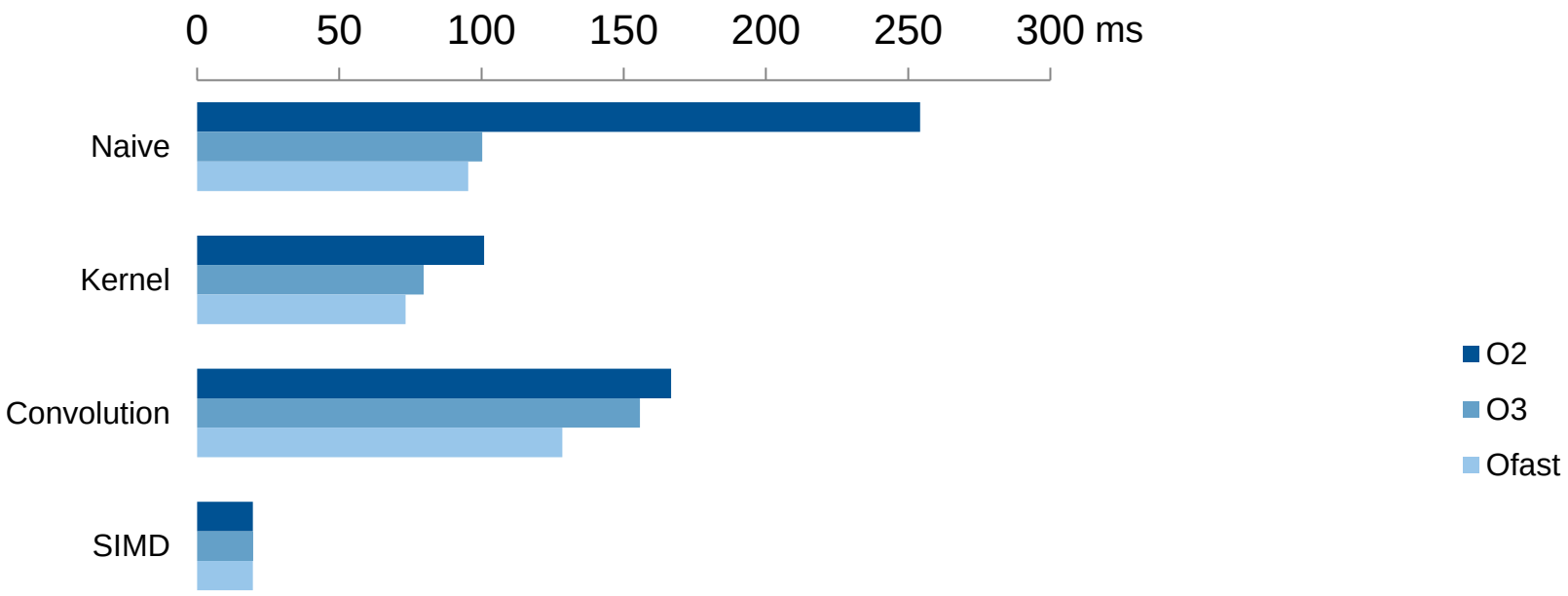
27% faster for Kernel unroll

22% faster for Convolution

no improvement for SIMD

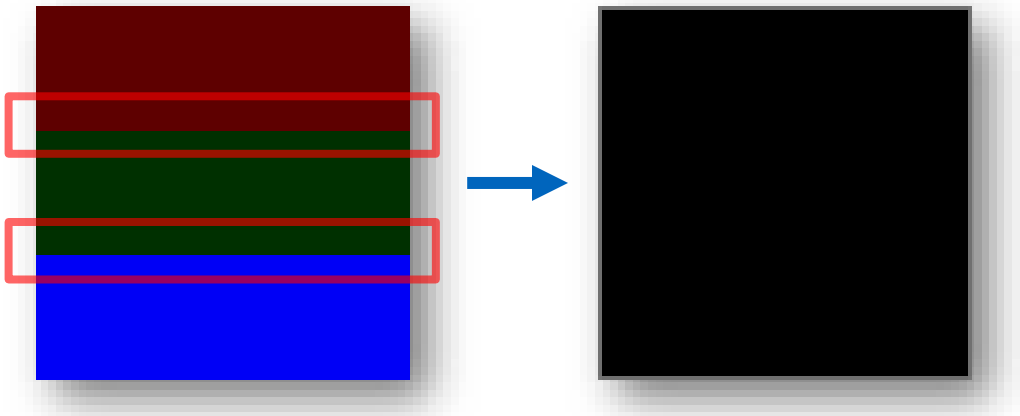
AMD Ryzen 5 4680U, 2.20 GHz, 8 GB RAM, compiled with -Ofast

Compiler optimization

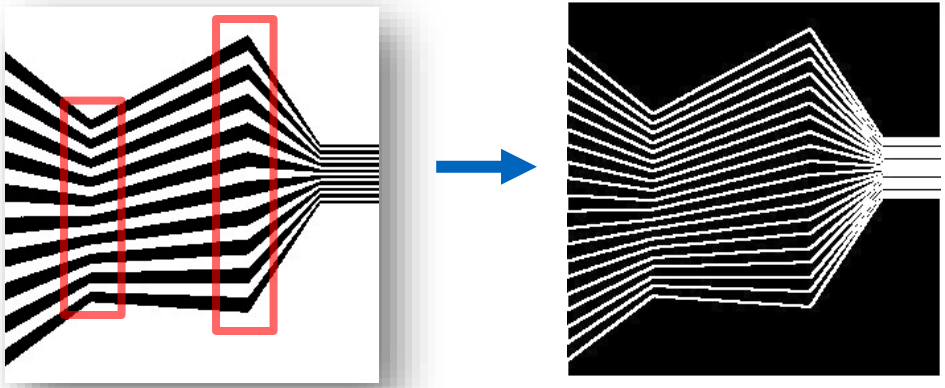


AMD Ryzen 5 4680U, 2.20 GHz, 8 GB RAM, compiled with -O2

General limitiations of the Sobel Filter



only works, if numeric representation of colour values differ



only colour based and very local, no global perception

Code optimization

- Microoptimization not very useful, C compiler goes a long way
- SIMD reduced computation time by a lot
- Larger Kernels for more accurate results
- Threads as another possible improvement